

Reproducing LightMem: Naive RAG Is Just as Good for Memory Management

Yongjie Zhou

The University of Queensland
Brisbane, QLD, Australia
yongjie.zhou@uq.edu.au

Bevan Koopman

The University of Queensland & CSIRO
Brisbane, QLD, Australia
bevan.koopman@csiro.au

Shuai Wang

The University of Queensland
Brisbane, QLD, Australia
shuai.wang2@uq.edu.au

Guido Zuccon

The University of Queensland & Google
Brisbane, QLD, Australia
g.zuccon@uq.edu.au

Abstract

Long-term conversational agents require access to information from earlier interactions, such as a user’s preferences, past requests, or previously mentioned facts. Repeatedly providing the full dialogue history can be expensive as conversations grow, so many memory approaches instead transform past interactions into compact entries that can be retrieved when needed. LightMem is a recent lightweight memory-management approach that reports strong effectiveness while maintaining relatively low construction cost. However, it still relies on a separate constructed memory representation and is evaluated with only one retriever, leaving unclear how sensitive its results are to retriever choice and whether memory construction discards answer-relevant information.

In this study, we reproduce LightMem and compare it with Naive RAG, which retrieves directly from raw user turns. We recover LightMem’s main configuration trend, but find that retriever choice is a major source of performance variation: changing only the retriever over a fixed LightMem store shifts answer accuracy from 58.1% to 75.5%. Constructed memories also do not consistently outperform raw-turn retrieval. Naive RAG generally performs better at matched retrieval depths, whereas LightMem performs better mainly under tight answering-token budgets. Oracle evaluation further shows that memory construction removes some answer-relevant information.

Overall, LightMem offers a context-efficiency trade-off rather than a general advantage over Naive RAG. Its value depends on the retriever and available token budget, motivating future work on retrieval, reranking, query formulation, and their interaction with raw and constructed memory representations. Our code is available at <https://github.com/ielab/Reproducing-LightMem>.

Keywords

LLM agent memory; Retrieval-augmented generation; Conversational retrieval; Memory retrieval

1 Introduction

Large language model (LLM) agents require memory to maintain continuity across long-term interactions, such as recalling a user’s preferences or earlier requests across conversations. A straightforward approach is to repeatedly provide the full interaction history to the model. However, this becomes increasingly expensive as the

history grows and may eventually exceed the model’s context window. Memory systems therefore store past interactions and retrieve only the information relevant to the current query.

Many existing systems, including A-MEM [28] and MemoryOS [9], do not retrieve directly from the original dialogue. Instead, they use LLMs to transform past interactions into structured memory entries by extracting, summarising, linking, or updating information as new interactions arrive. Although these representations may be more compact and organised, constructing them can require hundreds or thousands of LLM calls over a long history, introducing substantial computational cost and latency.

LightMem [4] is designed to reduce this construction overhead. It filters tokens from raw dialogue turns and groups related turns by topic, allowing multiple interactions to be processed together during memory construction. This reduces the number of LLM calls compared with systems that construct or update memories for individual interactions.

However, LightMem still transforms the original dialogue into a separate memory representation before retrieval. This transformation introduces an important trade-off. Constructed memories may be shorter and easier to retrieve, but summarising, merging, and updating dialogue turns may also cause **information loss**. End-to-end answer accuracy alone cannot determine whether a failure arises during **memory construction** or **retrieval**. Moreover, LightMem is evaluated using a single retriever, leaving its **sensitivity to retriever choice** unclear. More fundamentally, it remains uncertain whether **constructed memories outperform direct retrieval over raw dialogue turns**.

We investigate these questions through a reproduction and controlled evaluation of LightMem against a raw-turn retrieval-augmented generation baseline, which we refer to as **Naive RAG** [7, 13]. We first reproduce LightMem under its reported configurations. We then hold the constructed memory store fixed and evaluate 11 sparse, dense, and hybrid retrievers. Finally, we compare LightMem and Naive RAG using the same retrievers under matched retrieval depths and matched answering-token budgets. We also use oracle conditions to separate retrieval errors from information lost during memory construction. We organise the study around the following research questions:

RQ1: To what extent can LightMem be reproduced? We compare reproduced results with those reported in the original study in terms of answer accuracy and memory-construction efficiency.

RQ2: How does retriever choice affect LightMem? We hold the memory store and answer-generation settings fixed and evaluate a range of sparse, dense, and hybrid retrieval methods.

RQ3: Does memory construction provide an advantage over retrieving raw dialogue turns? We compare LightMem with Naive RAG under matched retrieval depths and matched answering-token budgets, and use oracle evidence to distinguish retrieval limitations from information loss during construction.

Our results recover LightMem’s main configuration trend and show that it can outperform the original Naive RAG and full-context baselines with the retriever used in the original study. However, this advantage is not consistent across configurations, and retriever choice is a major source of variation. Strong retrieval over raw turns often matches or exceeds retrieval over constructed memories (LightMem) without requiring a separate construction stage. Oracle comparisons further show that construction can remove answer-relevant information, although compact memories remain useful under tight answering-token budgets. These findings suggest that future work should not focus only on memory construction, as retrieval remains a major bottleneck in long-term memory systems. Improving retrievers, reranking, and query formulation, and understanding how they interact with raw and constructed memory representations, may be as important as designing more elaborate memory-construction methods.

2 Related Work

2.1 Memory Systems for LLM Agents

Memory systems allow LLM agents to retain and reuse information beyond the current context. Early work such as MemGPT [15] manages conversation history through a hierarchy of active and external memory. More recent systems, including A-MEM [28] and MemoryOS [9], transform interactions into structured memory entries that can be linked, updated, and consolidated over time [2, 14, 18, 20, 23, 31]. Although these systems differ in how memories are represented and maintained, they often rely on repeated LLM calls during memory construction.

A simpler alternative is to store raw dialogue turns and retrieve relevant turns directly at answering time, as in retrieval-augmented generation (RAG) [7, 13]. This avoids a separate construction stage and preserves the original interaction content, but places greater responsibility on the retriever. Our work compares these two designs: retrieval over constructed memories and retrieval over raw dialogue turns.

2.2 Retrieval Methods

Sparse retrievers such as BM25 [21] rank documents using lexical overlap, while learned sparse methods such as SPLADE [5, 6, 12] learn term weights and expansions. Dense retrievers encode queries and documents as vectors and rank them by semantic similarity [10, 19]. Hybrid and fusion methods instead combine lexical and dense signals [3, 24]. Because memory construction changes the wording and granularity of the indexed text, retrievers may behave differently over constructed memories and raw dialogue turns [16].

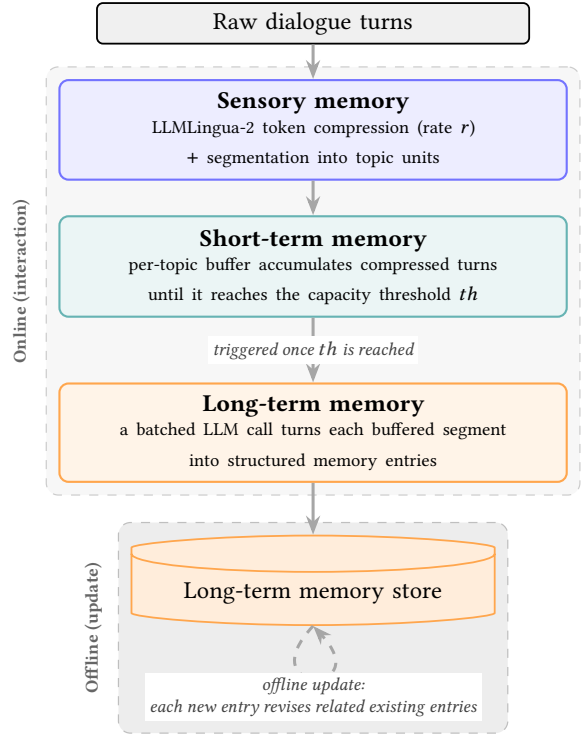


Figure 1: The original LightMem pipeline. Dialogue turns are compressed and grouped into topic-based segments before being summarised into long-term memory entries. Initial memory construction is performed online, while updates to existing long-term memories are performed offline.

2.3 Evaluating LLM Memory Systems

LLM memory systems are commonly evaluated through end-to-end answer accuracy on benchmarks such as LongMemEval [8, 26]. However, this metric combines errors from memory construction, retrieval, and answer generation [27]. An incorrect answer may arise because relevant information was lost during construction [1], missed during retrieval, or not used correctly by the generator [25]. Comparisons are further complicated by differences in retrieval-unit length, since one constructed memory may represent several raw dialogue turns. We address these issues through matched retrieval depths, matched answering-token budgets, and oracle evaluation.

3 Reproducing LightMem

LightMem [4] is a lightweight long-term memory approach for LLM agents. It compresses and groups dialogue turns before summarising them into long-term memory entries, which are later retrieved to answer new questions. Figure 1 provides an overview of its sensory, short-term, and long-term memory stages.

3.1 Memory Construction

LightMem uses *sensory memory* as the first processing stage for newly arriving dialogue turns. Rather than storing each turn directly, it applies LLMLingua-2 [17] to retain only a fraction of its

tokens. For a turn containing N tokens, the compressed turn contains approximately

$$N_{\text{comp}} \approx rN,$$

where $r \in (0, 1]$ is the compression rate. A larger r retains more of the original turn and applies less aggressive compression, while a smaller r removes more tokens. Each compressed turn is then assigned to a topic group and accumulated in the corresponding short-term memory buffer.

Let B denote the total number of compressed tokens currently stored in a topic buffer. Memory construction is triggered when

$$B \geq th,$$

where th is the short-term memory capacity threshold. A larger th allows more related content to accumulate before summarisation, while a smaller th triggers summarisation earlier and more frequently.

Once the threshold is reached, an LLM summarises the grouped turns into a long-term memory entry. Processing several related turns together avoids constructing a separate memory after every interaction. LightMem also performs an offline update stage that identifies related long-term entries and revises or consolidates them outside the online interaction process.

3.2 Memory Retrieval and Answering

At inference time, LightMem uses the current question as a query over the long-term memory store. The original study encodes both the question and each memory entry with the dense sentence-embedding model all-MiniLM-L6-v2 [22], ranks the entries by embedding similarity, and selects the top- k results. The retrieved memories are provided to the answer-generation model together with the current question.

3.3 Reproduction and Evaluation Framework

Our study reproduces the LightMem pipeline and extends its evaluation to examine memory construction and retrieval separately. Figure 2 summarises the framework. **LightMem** transforms raw dialogue turns into constructed memory entries before retrieval, whereas **Naive RAG** retrieves directly from the raw turns.

We first reproduce the original LightMem evaluation using its reported configurations, default retriever, and comparison baselines, and assess both answer accuracy and construction efficiency. We then fix the constructed memory store and evaluate a broader range of retrievers. Finally, we compare retrieval over constructed memories with retrieval over raw user turns under matched retrievers, retrieval depths, and answering-token budgets. Oracle conditions are used to distinguish retrieval limitations from information lost during memory construction.

The three research questions share the same dataset, generation model, core LightMem construction settings, and answer-evaluation protocol. Each RQ section therefore specifies only the additional settings required for that experiment.

Dataset. We evaluate on LongMemEval-S [26], a benchmark for long-term conversational memory. It contains 500 questions across

Table 1: LightMem construction settings used in our reproduction. We evaluate the paired configurations $(r, th) \in \{(0.4, 768), (0.6, 768), (0.8, 1024)\}$; all other settings follow the released implementation.

Parameter	Value
<i>Pre-compression and topic segmentation</i>	
pre_compress	true
compressor	llmlingua-2
rate (r)	{0.4, 0.6, 0.8}
topic_segment	true
precomp_topic_shared	true
<i>Memory construction</i>	
messages_use	user_only
metadata_generate	true
text_summary	true
stm_max_tokens (th)	{768, 1024}
max_tokens	16000
<i>Offline update (OP-UPDATE)</i>	
update	offline
update_sim_threshold	0.8
update_queue_top_k	20
update_queue_keep_top_n	10

six categories: single-session-user, single-session-preference, single-session-assistant, multi-session, knowledge-update, and temporal-reasoning. It also includes abstention questions for which no supporting evidence exists, with gold evidence annotated at the level of raw dialogue turns. Following the original study, we exclude the 56 single-session-assistant questions because LightMem constructs memories only from user turns, leaving 444 questions for all experiments.

Generation model. All systems use Qwen3-30B-A3B-Instruct-2507 [29] for answer generation. LightMem also uses the same model for memory summarisation and offline updates. We serve the model locally with vLLM [11] (v0.22.1)¹ and run it on H100 GPUs.

LightMem construction settings. The shared LightMem construction settings are reported in Table 1. RQ-specific choices, including the selected (r, th) configuration, retriever, retrieval depth, and comparison condition, are described in the corresponding setup sections.

Answer evaluation. Following LightMem [4] and LongMemEval-S [26], we evaluate generated answers using an LLM judge. We use the fixed gpt-5.5-2026-04-23 snapshot² to keep judging consistent across all runs, together with the category-specific evaluation prompts provided in Table 2.³ For abstention questions, an answer is counted as correct only when the model declines to answer.

¹<https://github.com/vllm-project/vllm>

²The original LightMem study uses gpt-4o-mini as the judge. We use a stronger model because we expect it to provide more reliable evaluations. Re-evaluating our outputs with gpt-4o-mini produces similar overall conclusions; these results are reported in Appendix B.

³LongMemEval evaluation prompt source code: https://github.com/xiaowu0162/LongMemEval/blob/main/src/evaluation/evaluate_qa.py

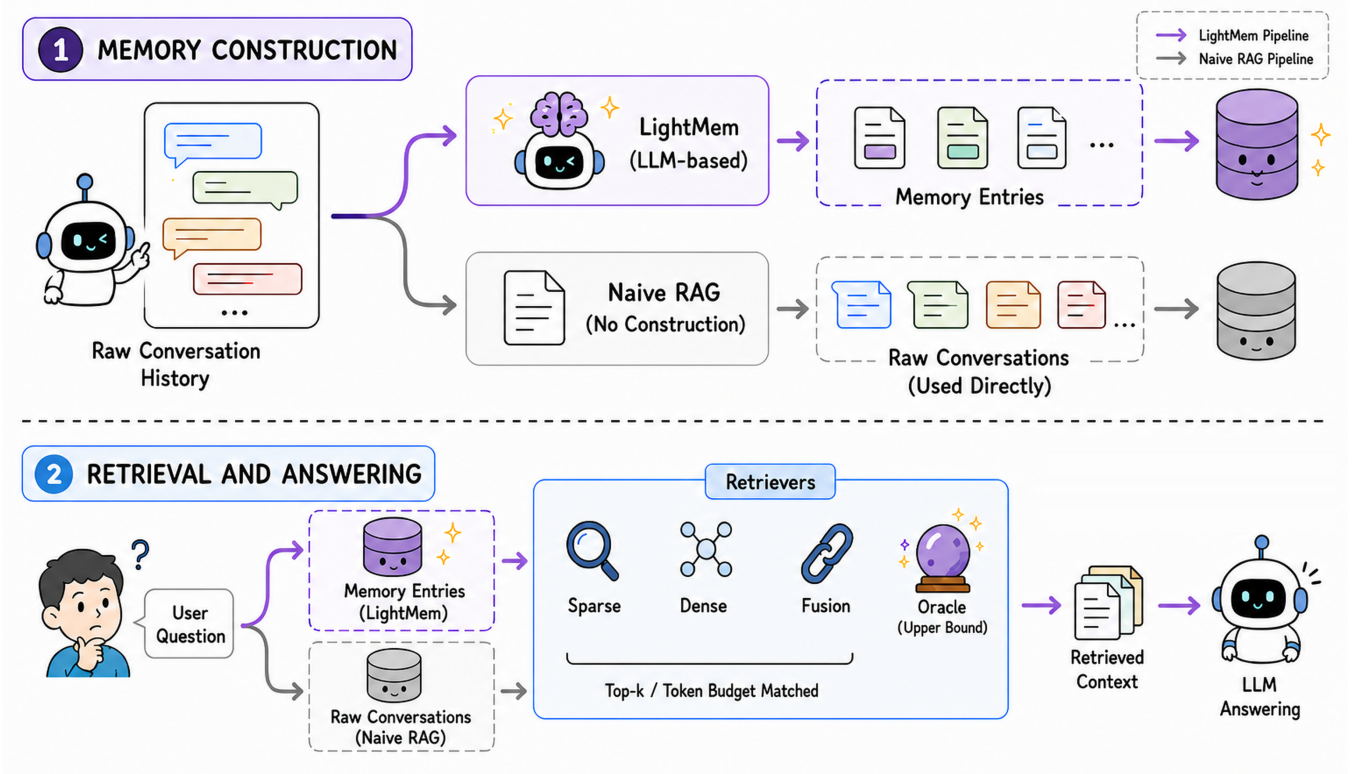


Figure 2: Overview of our evaluation framework. LightMem retrieves from constructed memory entries, whereas Naive RAG retrieves directly from raw user turns. We compare the two representations across retrievers, retrieval depths, answering-token budgets, and oracle conditions.

4 RQ1: Can We Reproduce LightMem?

4.1 Setup

We compare LightMem with the two baselines used in the original study. *Full-context* provides all raw user turns to the answer-generation model without retrieval, while *Naive RAG* [7, 13] retrieves directly from the raw user turns without constructing memory entries. We evaluate the three reported LightMem configurations:

$$(r=0.4, th=768), \quad (r=0.6, th=768), \quad (r=0.8, th=1024).$$

Offline updating is enabled in all cases, and the remaining construction settings are listed in Table 1.

For both LightMem and Naive RAG, we use all-MiniLM-L6-v2 [22], matching the original study, and retrieve the top-10 items. The original paper does not report the retrieval depth, so we follow the default setting in the released codebase.

We assess reproducibility using answer accuracy, construction-token usage, and construction LLM calls. Because our judge differs from that used in the original study, we focus on relative method ordering and trends across configurations rather than exact agreement in absolute accuracy.

Construction cost includes online summarisation and offline OP-UPDATE. We report construction tokens (CONSTR.) and construction LLM calls (CONSTR. CALLS) as totals over these two stages, which

the original study reports separately. Answering-token usage is not reported in the original paper and is therefore included as an additional analysis. Construction costs are measured per ingested sample, while answering tokens are measured per graded question.

4.2 Results

Table 3 compares our reproduction with the results reported for Qwen3-30B-A3B-Instruct-2507 [29] in the original LightMem study.

Original finding: LightMem outperforms Full-context and Naive RAG, with accuracy improving across the three reported configurations [4].

We reproduce the configuration trend, but not the absolute accuracy values. In both studies, accuracy increases across the three configurations, and $(r=0.8, th=1024)$ performs best. Our results therefore recover the ordering reported in the original study, despite using a different LLM judge.

The comparison with the baselines is less consistent. In our runs, $(r=0.6, th=768)$ and $(r=0.8, th=1024)$ achieve 68.2% and 70.7% accuracy, respectively, exceeding Full-context at 60.8% and Naive RAG at 67.3%. By contrast, $(r=0.4, th=768)$ reaches 57.0% and underperforms both baselines. We therefore reproduce LightMem’s

Table 2: Category-specific LLM-judge prompts for answer evaluation, taken from LongMemEval [26]. Each prompt returns a yes/no verdict, and placeholders in { . . . } are filled at evaluation time.

single-session-user / multi-session I will give you a question, a correct answer, and a response from a model. Please answer yes if the response contains the correct answer. Otherwise, answer no. If the response is equivalent to the correct answer or contains all the intermediate steps to get the correct answer, you should also answer yes. If the response only contains a subset of the information required by the answer, answer no. Question: {question} Correct Answer: {answer} Model Response: {response} Is the model response correct? Answer yes or no only.	abstention (any category) I will give you an unanswerable question, an explanation, and a response from a model. Please answer yes if the model correctly identifies the question as unanswerable. The model could say that the information is incomplete, or some other information is given but the asked information is not. Question: {question} Explanation: {explanation} Model Response: {response} Does the model correctly identify the question as unanswerable? Answer yes or no only.
knowledge-update I will give you a question, a correct answer, and a response from a model. Please answer yes if the response contains the correct answer. Otherwise, answer no. If the response contains some previous information along with an updated answer, the response should be considered as correct as long as the updated answer is the required answer. Question: {question} Correct Answer: {answer} Model Response: {response} Is the model response correct? Answer yes or no only.	single-session-preference I will give you a question, a rubric for desired personalized response, and a response from a model. Please answer yes if the response satisfies the desired response. Otherwise, answer no. The model does not need to reflect all the points in the rubric. The response is correct as long as it recalls and utilizes the user's personal information correctly. Question: {question} Rubric: {rubric} Model Response: {response} Is the model response correct? Answer yes or no only.
temporal-reasoning I will give you a question, a correct answer, and a response from a model. Please answer yes if the response contains the correct answer. Otherwise, answer no. If the response is equivalent to the correct answer or contains all the intermediate steps to get the correct answer, you should also answer yes. If the response only contains a subset of the information required by the answer, answer no. In addition, do not penalize off-by-one errors for the number of days. If the question asks for the number of days/weeks/months, etc., and the model makes off-by-one errors (e.g., predicting 19 days when the answer is 18), the model's response is still correct. Question: {question} Correct Answer: {answer} Model Response: {response} Is the model response correct? Answer yes or no only.	

Table 3: Reproduction results on LongMemEval-S over 444 graded questions. We compare the values reported by LightMem [4] with our reproduction for answer accuracy (ACC), memory-construction tokens (Constr.), and construction LLM calls (Constr. calls). Answering tokens (Ans.) are reported only for our runs because they are unavailable in the original study. Construction costs include both online summarisation and offline OP-UPDATE. Token counts are in thousands, and the highest accuracy in each block is shown in bold.

Method	Original (from paper)			Ours (reproduced)			
	ACC (%)	Constr.	Constr. calls	ACC (%)	Constr.	Constr. calls	Ans.
Full-context	54.8	0	0	60.8	0	0	18.84
Naive RAG	60.8	0	0	67.3	0	0	1.03
LightMem ($r=0.4$, $th=768$)	62.3	144.16	192.56	57.0	72.68	64.69	0.72
LightMem ($r=0.6$, $th=768$)	65.1	135.43	172.90	68.2	106.90	104.52	0.67
LightMem ($r=0.8$, $th=1024$)	67.3	146.42	177.80	70.7	119.88	117.62	0.66

advantage for the two higher-performing configurations, but not for every reported setting.

Reproduction target: LightMem's reported construction-token usage and number of construction LLM calls under the three configurations [4].

We reproduce the presence of substantial construction overhead, but not the reported absolute costs. The original study reports 135.43–146.42k construction tokens and 172.90–192.56 construction LLM calls per sample. Our corresponding runs require 72.68–119.88k tokens and 64.69–117.62 calls, consistently below the reported values.

The source of this quantitative discrepancy is unclear. Our results nevertheless follow the expected cost pattern: a larger compression rate r retains more tokens, with ($r=0.8, th=1024$) incurring the highest construction cost and ($r=0.4, th=768$) the lowest. The original results do not show this ordering as clearly. Construction cost also depends on implementation behaviour, including how many memory entries are created and when offline updates are triggered. Differences in these details may therefore explain the mismatch. Because they are not fully specified in the paper, we follow the released implementation.

Table 3 additionally reports answering-token usage, which reflects the online cost at answering time. At top-10, LightMem uses 0.66–0.72k answering tokens per question, compared with 1.03k for Naive RAG and 18.84k for Full-context. LightMem therefore reduces the context passed to the answer-generation model and provides a clear online cost advantage. This saving, however, is achieved only after incurring the substantial upfront cost of memory construction.

Overall: We reproduce LightMem’s configuration ordering and show that it can outperform Full-context and Naive RAG with the retriever used in the original study, while also reducing online answering-token usage. However, this advantage does not hold for every configuration, and the reported absolute accuracy and construction-cost values are not reproduced.

5 RQ2: How Does Retrieval Affect LightMem?

5.1 Setup

To isolate the effect of retrieval, we fix the LightMem store constructed with the best-performing configuration from RQ1, $r=0.8$ and $th=1024$, and vary only the retriever used at answering time. Each retriever returns the top-10 memory entries, which are provided to the same answer-generation model.

The original study used only all-MiniLM-L6-v2 [22]. We extend the evaluation to 11 retrievers from three families:

- **Sparse retrieval:** BM25 [21] provides a lexical baseline, while SPLADE-v3 [12] and PromptReps-sparse [32] use learned sparse representations.
- **Dense retrieval:** We evaluate all-MiniLM-L6-v2, PromptReps-dense, and Qwen3-Embedding [30] at 0.6B, 4B, and 8B scales.
- **Hybrid retrieval:** We evaluate PromptReps-hybrid and score-fusion [24] variants combining BM25 with either all-MiniLM-L6-v2 or Qwen3-Embedding-0.6B.

We report Recall@10 and downstream answer accuracy. LongMemEval-S provides gold evidence at the level of raw dialogue turns rather than constructed memories. We therefore treat a LightMem entry as relevant if it was constructed or updated from a turn marked has_answer. Recall is computed over the 422

questions with gold evidence; abstention questions are evaluated only by answer accuracy.

We also include an oracle-reference condition in which all memory entries associated with has_answer turns are provided directly to the answer-generation model. This removes retrieval error under the available relevance mapping. It is not a strict upper bound, however, because entries not linked to a labelled turn may still contain useful context.

5.2 Results

Figure 3 relates Recall@10 to answer accuracy, while Table 4 reports results by question type and significance tests against the default retriever and the oracle-reference condition.

Finding 1: Retriever choice substantially affects the effectiveness of LightMem.

With the memory store fixed, Recall@10 ranges from 0.390 with BM25 to 0.587 with Qwen3-Embedding-4B, while answer accuracy ranges from 58.1% to 75.5%. Since the memory representation and answer-generation model remain unchanged, this variation is attributable to how the same constructed memories are retrieved.

Qwen3-Embedding-4B achieves the highest recall and answer accuracy. Its accuracy of 75.5% is 2.2 points below the oracle-reference result of 77.7%, and this difference is not statistically significant. This does not establish equivalence to the oracle, but suggests that a strong retriever can recover much of the answerable information available in the fixed memory store. Remaining errors may arise from information lost during construction or from failures in answer generation.

Finding 2: Retriever choice matters most when evidence is distributed across sessions or expressed indirectly.

The largest category-level differences occur for multi-session and single-session-preference questions, and the spread remains substantial even after excluding BM25. This suggests that these categories benefit most from stronger semantic retrieval, likely because the required evidence is distributed across sessions or phrased differently from the query.

Knowledge-update, single-session-user, and temporal-reasoning questions are more stable among non-BM25 retrievers, suggesting that their remaining errors may arise more from memory construction or answer generation than from retrieval. Some retrievers also exceed the oracle-reference result for single-session-user questions, confirming that the has_answer-based mapping does not capture every useful memory entry.

Abstention questions behave differently because they contain no gold evidence. Retrieved but irrelevant memories may encourage the model to answer when it should abstain.

Finding 3: Fusion provides no consistent benefit over the corresponding base retrievers.

The fusion variants do not improve on their corresponding base retrievers. Adding BM25 lowers both recall and answer accuracy for the MiniLM- and Qwen3-based systems, with the recall reduction for MiniLM+BM25 statistically significant relative to the default

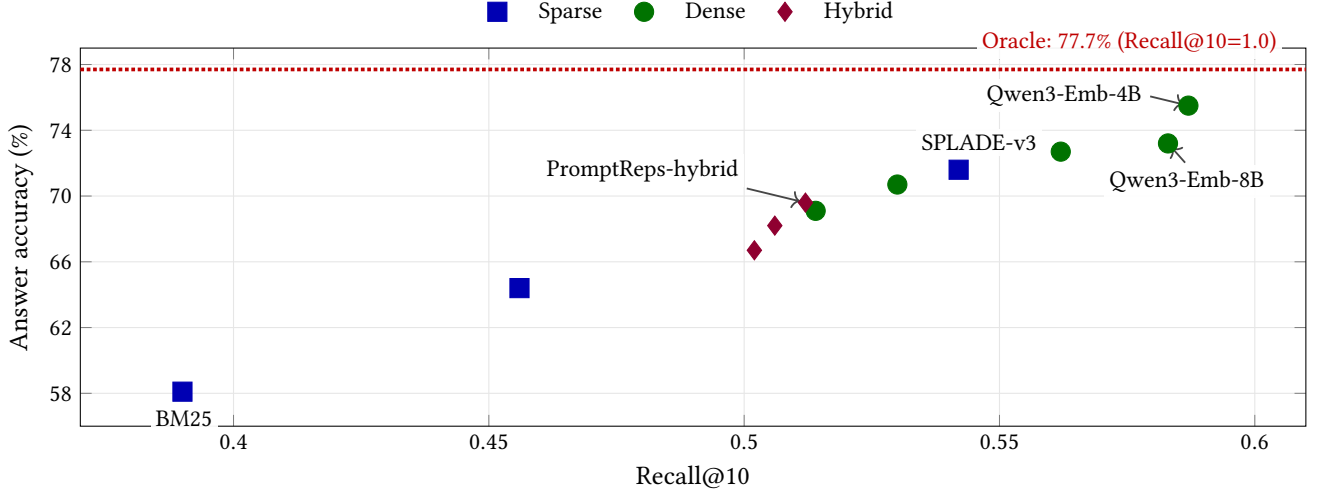


Figure 3: Relationship between Recall@10 and answer accuracy for LightMem at top-10. Each point represents one retriever; colours and markers indicate retriever families, and selected retrievers are labelled. The dotted horizontal line shows the oracle-reference accuracy obtained by providing all memory entries linked to has_answer turns to the generator. The oracle recall is 1.0 and lies outside the displayed x-axis range.

Table 4: Per-question-type Recall@10 / answer accuracy (%); *Abst.* is accuracy only because abstention questions have no gold evidence. Among real retrievers, the best and second-best values in each column are highlighted, and the **shaded row** marks the LightMem default, all-MiniLM-L6-v2. Superscripts indicate a significant difference from the default (a) or from Oracle (b), using a paired *t*-test for Recall@10 and McNemar’s exact test for accuracy. Tests are computed over the questions in each column, with Bonferroni correction applied within each column and anchor family at $p < 0.05$. Recall@10 is not tested against Oracle, where it is 1.0 by construction.

	Retriever	Know-upd.	Multi-sess.	Single-pref.	Single-user	Temporal	Abst.	Overall
Sparse	BM25	0.467 ^a / 76.4	0.300 ^a / 38.0 ^{ab}	0.257 ^a / 33.3 ^{ab}	0.546 ^a / 84.4	0.382 ^a / 61.4	50.0	0.390 ^a / 58.1 ^{ab}
	SPLADE-v3	0.595 / <u>81.9</u>	0.475 / 66.1	0.641 / 66.7 ^b	0.670 / <u>92.2</u>	0.487 / 68.5	43.3 ^b	0.542 / 71.6
	PromptReps-sparse	0.519 / 77.8	0.398 ^a / 46.3 ^b	0.333 ^a / 50.0 ^b	0.603 / 89.1	0.429 / 67.7	<u>53.3</u>	0.456 ^a / 64.4 ^{ab}
Dense	all-MiniLM-L6-v2	0.571 / 81.9	0.464 / 59.5	0.607 / 70.0	0.691 / 87.5	0.474 / 71.7	50.0	0.530 / 70.7
	Qwen3-Emb-0.6B	0.600 / <u>81.9</u>	0.520 ^a / <u>68.6</u>	0.699 / <u>73.3</u>	0.703 / 93.8	0.486 / 67.7	43.3	0.562 ^a / 72.7
	Qwen3-Emb-4B	0.637^a / 83.3	<u>0.523^a</u> / 70.2	0.747 / 80.0	0.699 / 93.8	0.531^a / <u>72.4</u>	46.7	0.587^a / 75.5
	Qwen3-Emb-8B	<u>0.618</u> / 80.6	0.532^a / 66.9	0.733 / <u>73.3</u>	0.712 / 90.6	<u>0.510</u> / 73.2	43.3	<u>0.583^a</u> / <u>73.2</u>
	PromptReps-dense	0.541 / 79.2	0.456 / 57.9	<u>0.737</u> / 70.0	0.656 / 89.1	0.431 / 66.9	56.7	0.514 / 69.1 ^b
	PromptReps-hybrid	0.559 / 79.2	0.460 / 57.0	0.490 / 70.0	0.644 / 89.1	0.470 / 71.7	46.7	0.512 / 69.6 ^b
Hybrid	MiniLM+BM25	0.556 / 80.6	0.434 / 54.5	0.455 ^a / 56.7 ^b	0.653 / 85.9	0.476 / 70.9	33.3 ^b	0.502 ^a / 66.7 ^b
	Qwen3-0.6B+BM25	0.566 / 79.2	0.451 / 57.9	0.421 ^a / 50.0 ^b	0.634 / <u>92.2</u>	0.479 / 70.1	43.3 ^b	0.506 / 68.2 ^b
	Oracle	1.000 / 77.8	1.000 / 69.4	1.000 / 96.7	1.000 / 87.5	1.000 / 75.6	80.0	1.000 / 77.7

retriever. PromptReps-hybrid performs only marginally better than PromptReps-dense and does not show a clear end-to-end advantage. Overall, combining lexical and semantic signals provides no consistent benefit over the fixed LightMem store in our evaluation.

Overall: Retriever choice has a substantial effect on LightMem. Dense and learned sparse retrievers generally perform better than BM25, with the largest benefits observed for multi-session and preference questions, and the best retriever approaching the oracle-reference condition. Combining BM25 with dense retrieval, however, provides no consistent benefit.

Table 5: Matched answering-token budgets for RQ3. For each Naive RAG setting, the LightMem retrieval depth is selected to approximate its mean answering-context length across the 11 retrievers.

Token budget	Naive RAG		LightMem		Cost ratio
	top- k	tok/q	top- k	tok/q	
~330 tok/q	3	330	6	329	1.00×
~500 tok/q	5	503	10	498	0.99×
~935 tok/q	10	946	20	923	0.98×

6 RQ3: Is Memory Construction Worth Its Cost?

6.1 Setup

RQ3 compares two representations of the same conversation history. LightMem retrieves from memories constructed using its best-performing configuration from RQ1, ($r=0.8, th=1024$), whereas Naive RAG retrieves directly from raw user turns. Constructed memories provide shorter retrieval units, but may omit details retained in the original dialogue.

We compare the two systems using the same 11 retrievers as in RQ2 under two settings. First, in the **matched retrieval depth** comparison, both systems use the same retriever and retrieve $k \in \{3, 5, 10\}$ items. This tests whether a constructed memory is more useful than a raw turn when the number of retrieved units is fixed.

Second, in the **matched answering-token budget** comparison, we approximately match the context provided to the answer-generation model. Because constructed memories are generally shorter than raw turns, equal retrieval depths give Naive RAG a larger answering context. We therefore measure the mean context length at each depth and select the closest pairs across the two representations. As shown in Table 5, LightMem at top-6, top-10, and top-20 approximately matches Naive RAG at top-3, top-5, and top-10, respectively.

Finally, we evaluate an **oracle condition** for each representation. Naive RAG receives the gold raw turns, while LightMem receives the memories constructed from those turns. Removing retrieval error allows us to test how much answer-relevant information is preserved during memory construction. We also compare LightMem’s construction cost with its answering-token savings.

6.2 Results

Figure 4 compares LightMem and Naive RAG under matched retrieval depths and answering-token budgets. Each cell reports the accuracy difference $\Delta = \text{LightMem} - \text{Naive RAG}$, such that positive values favour LightMem.

Finding 1: When retrieval depth is matched, Naive RAG generally outperforms LightMem.

At the same retriever and retrieval depth, most comparisons favour Naive RAG. Its average advantage is largest at top-5 and remains similar at top-10, showing that constructed memories do not provide a general benefit when both systems retrieve the same number of units.

Table 6: Oracle accuracy and memory-construction cost for RQ3. Oracle accuracy is measured using gold evidence, removing retrieval error. Construction cost is reported per ingested sample, while answering tokens are reported per question.

Metric	Naive RAG	LightMem
Oracle accuracy (%)	89.0	77.7
Construction tokens	0	119,884
Construction LLM calls	0	117
Answering tokens / question	~903	~529

The result nevertheless depends on the retriever. LightMem consistently performs better with all-MiniLM-L6-v2 and PromptReps-dense, while Qwen3-Embedding-4B remains close to parity. These cases suggest that some retrievers interact more favourably with compact memory entries. For most other retrievers, however, retrieving the original dialogue is more effective than retrieving its constructed representation.

Finding 2: LightMem is most useful under tight answering-token budgets, but its advantage disappears as the budget increases.

The matched-budget comparison changes the result at low context limits. At approximately 330 tokens per question, LightMem performs better for 8 of the 11 retrievers and leads by 5.5 accuracy points on average. Its mean advantage falls to 2.2 points at approximately 500 tokens and becomes a small disadvantage at approximately 935 tokens.

LightMem’s compact entries allow more retrieved units to fit within a restricted context, improving evidence coverage when the answering budget is tight. As more context becomes available, however, Naive RAG can retrieve sufficient raw evidence without discarding details during construction.

This benefit remains retriever-dependent. It is most consistent with all-MiniLM-L6-v2 and PromptReps-dense, while gains for most other retrievers diminish or reverse at larger budgets. The usefulness of memory construction therefore depends partly on how the retriever interacts with the resulting representation.

Finding 3: Memory construction loses answer-relevant information while introducing substantial upfront cost.

Table 6 summarises the oracle comparison, construction cost, and answering-token usage. Naive RAG reaches 89.0% accuracy when given the gold raw turns, compared with 77.7% for LightMem when given the corresponding constructed memories. Because retrieval error is removed in both conditions, this 11.3-point gap indicates that memory construction discards some answer-relevant information. Consistent with this result, Naive RAG with SPLADE-v3 at top-10 reaches 78.8%, already exceeding the LightMem oracle-reference accuracy. Although the LightMem oracle is not a strict upper bound, these results suggest that improving retrieval over the fixed memory store alone may not recover all of the information available in the raw dialogue.

LightMem instead provides an online efficiency benefit. It reduces the mean answering context from approximately 903 to 529

Δ color: ■ LightMem better ■ near parity ■ Naive RAG better ■ darker = larger gap

		(a) Matched retrieval depth			(b) Matched answering-token budgets		
Sparse	Retriever	top-3	top-5	top-10	~330t	~500t	~935t
	BM25	-7.7	-10.8	-10.6	-1.6	-4.7	-3.2
	SPLADE-v3	-7.0	-6.9	-7.2	+4.7	-0.2	-2.2
	PromptReps-sparse	-8.5	-6.7	-5.0	-1.1	+0.9	-2.3
Dense	all-MiniLM-L6-v2	+8.8	+4.9	+3.4	+17.8	+11.2	+7.5
	Qwen3-Emb-0.6B	-3.4	-6.5	-5.0	+2.7	-0.7	-1.8
	Qwen3-Emb-4B	+0.2	+1.3	-0.4	+7.4	+5.9	+0.7
	Qwen3-Emb-8B	-1.8	-2.9	-4.5	+5.6	+1.8	-2.9
	PromptReps-dense	+4.3	+1.4	+1.3	+14.7	+12.8	+5.2
Hybrid	PromptReps-hybrid	-1.3	-7.0	-6.5	+7.2	+0.2	-4.9
	MiniLM+BM25	-2.9	-8.1	-5.1	+5.4	-1.1	-0.2
	Qwen3-0.6B+BM25	-6.8	-11.5	-8.8	-2.3	-2.3	-6.3

Figure 4: Comparison between LightMem and Naive RAG. Each cell reports the answer-accuracy difference, $\Delta = \text{LightMem} - \text{Naive RAG}$, in points; positive values favour LightMem. Panel (a) matches retrieval depth, while panel (b) matches answering-token budgets. Retrievers are grouped into sparse, dense, and hybrid families.

tokens per question, saving about 374 tokens. Constructing the memory store, however, requires 119,884 tokens and approximately 117 LLM calls per ingested sample. Based on token usage alone, this upfront cost would be recovered only after roughly 321 questions over the same history. This estimate is favourable to LightMem because it excludes the latency and monetary cost of the additional construction calls.

Overall: Whether LightMem is worth its construction cost depends on the retriever and the available answering-token budget. With retrievers such as all-MiniLM-L6-v2 and PromptReps-dense, compact memories can be beneficial under tight budgets. With stronger raw-turn retrievers or larger context budgets, however, Naive RAG performs as well as or better than LightMem while avoiding construction overhead and retaining more answer-relevant information.

7 Conclusion and Discussion

Our reproduction recovers LightMem’s main configuration trend, although the absolute accuracy and construction-cost values differ from those originally reported. Beyond this reproduction result, our broader evaluation shows that retriever choice is a major source of variation in LightMem’s effectiveness. Stronger retrievers substantially improve performance over the same constructed memory store, while strong retrieval over raw user turns

often matches or exceeds LightMem. **LightMem therefore offers a conditional rather than general advantage over Naive RAG: its compact memories are beneficial mainly under tight answering-token budgets or with retrievers that interact well with the constructed representation, while otherwise adding substantial construction overhead and risking the loss of answer-relevant information.**

The main lesson is not that memory construction is unnecessary, but that its value cannot be assessed independently of retrieval. Constructed memories are best viewed as a compact auxiliary representation rather than a replacement for the raw interaction history. Preserving the original dialogue provides a faithful source of evidence, while summaries or structured memories may support efficient access when context is limited. Future work should study retrieval, reranking, and query formulation together with memory construction, and evaluate construction and retrieval separately so that improvements can be attributed to the correct stage of the system.

References

- [1] Ding Chen, Simin Niu, Kehang Li, Peng Liu, Xiangping Zheng, Bo Tang, Xinchu Li, Feiyu Xiong, and Zhiyu Li. 2025. Halumem: Evaluating hallucinations in memory systems of agents. *arXiv preprint arXiv:2511.03506* (2025).
- [2] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshray Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413* (2025).

- [3] Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. 2009. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*. 758–759.
- [4] Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, and Ningyu Zhang. 2026. Lightmem: Lightweight and efficient memory-augmented generation. In *The Fourteenth International Conference on Learning Representations*. <https://arxiv.org/abs/2510.18866>
- [5] Thibault Formal, Carlos Lassance, Benjamin Piwowarski, and Stéphane Clinchant. 2021. SPLADE v2: Sparse lexical and expansion model for information retrieval. *arXiv preprint arXiv:2109.10086* (2021).
- [6] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. 2021. Splade: Sparse lexical and expansion model for first stage ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2288–2292.
- [7] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* (2023).
- [8] Yuanzhe Hu, Yu Wang, and Julian McAuley. 2025. Evaluating memory in llm agents via incremental multi-turn interactions. *arXiv preprint arXiv:2507.05257* (2025).
- [9] Jiazhang Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. Memory os of ai agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 25961–25970.
- [10] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*. 6769–6781.
- [11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
- [12] Carlos Lassance, Hervé Déjean, Thibault Formal, and Stéphane Clinchant. 2024. SPLADE-v3: New baselines for SPLADE. *arXiv preprint arXiv:2403.06789* (2024).
- [13] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [14] Zhiyu Li, Shichao Song, Hanyu Wang, Simin Niu, Ding Chen, Jiawei Yang, Chenyang Xi, Huayi Lai, Jihao Zhao, Yezhaohui Wang, et al. 2025. Memos: An operating system for memory-augmented generation (mag) in large language models. *arXiv preprint arXiv:2505.22101* (2025).
- [15] Charles Packer, Vivian Fang, Shishir G Patil, Kevin Lin, Sarah Wooders, and Joseph E Gonzalez. 2023. MemGPT: towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560* (2023). <https://arxiv.org/abs/2310.08560>
- [16] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Xufang Luo, Hao Cheng, Dongsheng Li, Yuqing Yang, Chin-Yew Lin, H Vicky Zhao, Lili Qiu, et al. 2025. On memory construction and retrieval for personalized conversational agents. In *The Thirteenth International Conference on Learning Representations*.
- [17] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, et al. 2024. LlmLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of the Association for Computational Linguistics: ACL 2024*. 963–981.
- [18] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956* (2025).
- [19] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 3982–3992.
- [20] Alireza Rezaeadeh, Zichao Li, Wei Wei, and Yujia Bao. 2025. From isolated conversations to hierarchical schemas: Dynamic tree memory representation for llms. In *The Thirteenth International Conference on Learning Representations*, Vol. 2025.
- [21] Stephen Robertson and Hugo Zaragoza. 2009. *The probabilistic relevance framework: BM25 and beyond*. Vol. 4. Now Publishers Inc.
- [22] Sentence Transformers. [n. d.]. all-MiniLM-L6-v2. Hugging Face model repository. Accessed July 14, 2026. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
- [23] Zhen Tan, Jun Yan, I-Hung Hsu, Rujun Han, Zifeng Wang, Long Le, Yiwen Song, Yanfei Chen, Hamid Palangi, George Lee, et al. 2025. In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 8416–8439.
- [24] Shuai Wang, Shengyao Zhuang, and Guido Zuccon. 2021. Bert-based dense retrievers require interpolation with bm25 for effective passage retrieval. In *Proceedings of the 2021 ACM SIGIR international conference on theory of information retrieval*. 317–324.
- [25] Yuxin Wang, Paul Thomas, Zhiwei Yu, Yuan Gao, Saeed Hassanpour, Soroush Vosoughi, Robert Sim, and Nick Craswell. 2026. Memory Makes the Difference: Evaluating How Different Memory Roles Shape Conversational Agents. *arXiv preprint arXiv:2606.25361* (2026).
- [26] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. Longmemeval: Benchmarking chat assistants on long-term interactive memory. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=pZiyCaVuti>
- [27] Jianfei Xiao, Xiang Yu, Chengbing Wang, Wuqiang Zheng, Xinyu Lin, Kaining Liu, Hongxun Ding, Yang Zhang, Wenjie Wang, Fuli Feng, et al. 2026. AlpsBench: An LLM Personalization Benchmark for Real-Dialogue Memorization and Preference Alignment. *arXiv preprint arXiv:2603.26680* (2026).
- [28] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems* 38 (2025), 17577–17604.
- [29] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [30] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, et al. 2025. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176* (2025).
- [31] Wanjuan Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 19724–19731.
- [32] Shengyao Zhuang, Xueguang Ma, Bevan Koopman, Jimmy Lin, and Guido Zuccon. 2024. PromptReps: Prompting large language models to generate dense and sparse representations for zero-shot document retrieval. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 4375–4391.

A Detailed Values Behind Figure 4

Figure 4 reports each comparison only as a colour-coded difference $\Delta = \text{LightMem} - \text{NaiveRAG}$, so the two accuracies behind every cell are not visible.

Tables 7 and 8 list them in full, as LightMem / Naive RAG / Δ : the first for the matched-depth setting in panel (a), the second for the matched-token-budget setting in panel (b). Both use the same gpt-5.5 judge as the main text.

B Results Under the gpt-4o-mini Judge

Throughout the paper we judge answers with gpt-5.5, while the original LightMem study [4] uses the weaker gpt-4o-mini. A natural worry is that our findings simply reflect the stronger judge, so we re-graded every answer with gpt-4o-mini, leaving the prompts and all other settings as before.

This affects only the accuracy numbers; recall, token counts, and call counts are untouched. The tables and figures below repeat those in the main text under the weaker judge. The ranking of methods, and our main claim that Naive RAG keeps pace with LightMem, hold in every research question.

B.1 RQ1: Reproduction

Table 9 repeats the reproduction of Table 3 under gpt-4o-mini.

B.2 RQ2: Retriever Comparison

Table 10 gives the per-question-type breakdown, and Figure 5 the recall-accuracy plot.

B.3 RQ3: Memory Management

Figure 6 gives the controlled LightMem-vs-Naive-RAG comparison, and Table 11 the oracle ceiling.

Table 7: Answer accuracy at matched retrieval depth (RQ3), with the same retriever and top- k on both sides. Each cell gives *LightMem* / *Naive RAG* / Δ in points, where Δ = *LightMem* – *NaiveRAG*; a positive Δ means *LightMem* wins.

Retriever	Family	top-3	top-5	top-10
BM25	lexical	47.3 / 55.0 / -7.7	52.0 / 62.8 / -10.8	58.1 / 68.7 / -10.6
SPLADE-v3	learned sparse	57.0 / 64.0 / -7.0	64.9 / 71.8 / -6.9	71.6 / 78.8 / -7.2
all-MiniLM-L6-v2	dense	58.8 / 50.0 / +8.8	64.4 / 59.5 / +4.9	70.7 / 67.3 / +3.4
Qwen3-Emb-0.6B	dense	61.7 / 65.1 / -3.4	66.9 / 73.4 / -6.5	72.7 / 77.7 / -5.0
Qwen3-Emb-4B	dense	61.7 / 61.5 / +0.2	70.9 / 69.6 / +1.3	75.5 / 75.9 / -0.4
Qwen3-Emb-8B	dense	60.4 / 62.2 / -1.8	68.5 / 71.4 / -2.9	73.2 / 77.7 / -4.5
PromptReps (dense)	LLM sparse	50.9 / 46.6 / +4.3	57.7 / 56.3 / +1.4	69.1 / 67.8 / +1.3
PromptReps (sparse)	LLM sparse	50.7 / 59.2 / -8.5	56.8 / 63.5 / -6.7	64.4 / 69.4 / -5.0
PromptReps (hybrid)	hybrid	57.9 / 59.2 / -1.3	62.4 / 69.4 / -7.0	69.6 / 76.1 / -6.5
Fusion MiniLM+BM25	fusion	55.0 / 57.9 / -2.9	59.7 / 67.8 / -8.1	66.7 / 71.8 / -5.1
Fusion Qwen3-0.6B+BM25	fusion	56.5 / 63.3 / -6.8	59.0 / 70.5 / -11.5	68.2 / 77.0 / -8.8
Mean Δ	–	-2.4	-4.8	-4.4

Table 8: Answer accuracy at matched input cost (RQ3). To equalise the answering budget we pair *Naive RAG* top-3/5/10 with *LightMem* top-6/10/20, giving roughly 330/500/935 tokens. Each cell gives *LightMem* / *Naive RAG* / Δ in points, where Δ = *LightMem* – *NaiveRAG*; a positive Δ means *LightMem* wins. *LightMem*’s one-time memory-construction cost is not counted here.

Retriever	Family	~330t	~500t	~935t
BM25	lexical	53.4 / 55.0 / -1.6	58.1 / 62.8 / -4.7	65.5 / 68.7 / -3.2
SPLADE-v3	learned sparse	68.7 / 64.0 / +4.7	71.6 / 71.8 / -0.2	76.6 / 78.8 / -2.2
all-MiniLM-L6-v2	dense	67.8 / 50.0 / +17.8	70.7 / 59.5 / +11.2	74.8 / 67.3 / +7.5
Qwen3-Emb-0.6B	dense	67.8 / 65.1 / +2.7	72.7 / 73.4 / -0.7	75.9 / 77.7 / -1.8
Qwen3-Emb-4B	dense	68.9 / 61.5 / +7.4	75.5 / 69.6 / +5.9	76.6 / 75.9 / +0.7
Qwen3-Emb-8B	dense	67.8 / 62.2 / +5.6	73.2 / 71.4 / +1.8	74.8 / 77.7 / -2.9
PromptReps (dense)	LLM sparse	61.3 / 46.6 / +14.7	69.1 / 56.3 / +12.8	73.0 / 67.8 / +5.2
PromptReps (sparse)	LLM sparse	58.1 / 59.2 / -1.1	64.4 / 63.5 / +0.9	67.1 / 69.4 / -2.3
PromptReps (hybrid)	hybrid	66.4 / 59.2 / +7.2	69.6 / 69.4 / +0.2	71.2 / 76.1 / -4.9
Fusion MiniLM+BM25	fusion	63.3 / 57.9 / +5.4	66.7 / 67.8 / -1.1	71.6 / 71.8 / -0.2
Fusion Qwen3-0.6B+BM25	fusion	61.0 / 63.3 / -2.3	68.2 / 70.5 / -2.3	70.7 / 77.0 / -6.3
Mean Δ	–	+5.5	+2.2	-0.9

Table 11: GPT-4o-mini judge. RQ3: oracle ceiling and fixed memory-construction cost. Oracle accuracy uses gold evidence directly, removing retrieval error. Construction cost is per ingested sample. Oracle accuracy is re-graded by GPT-4o-mini; token and call counts are unchanged.

Metric	Naive RAG	LightMem
Oracle accuracy (%)	87.8	75.5
Construction tokens	0	119,884
Construction LLM calls	0	117
Answering tokens / question	~903	~529

Table 9: GPT-4o-mini judge. Reproduction results on LongMemEval-S with 444 graded questions. For each method we report answer accuracy (ACC), memory-construction tokens (Constr.), and construction LLM calls (Constr. calls), placing the values reported by the LightMem paper [4] alongside our reproduction runs; answering tokens (Ans.) are reported for our runs only, as they are not available in the original study. Both Constr. and Constr. calls are totals across the online summarisation and offline OP-UPDATE stages. Token counts are in thousands. The highest accuracy in each block is shown in bold. Answer accuracy is re-graded by GPT-4o-mini; the *Original (from paper)* block and all token/call counts are unchanged.

Method	Original (from paper)			Ours (reproduced)			
	ACC (%)	Constr.	Constr. calls	ACC (%)	Constr.	Constr. calls	Ans.
Full-context	54.8	–	–	56.5	0	0	18.84
Naive RAG	60.8	–	–	66.9	0	0	1.03
LightMem ($r=0.4$, $th=768$)	62.3	144.16	192.56	58.3	72.68	64.69	0.72
LightMem ($r=0.6$, $th=768$)	65.1	135.43	172.90	67.8	106.90	104.52	0.67
LightMem ($r=0.8$, $th=1024$)	67.3	146.42	177.80	70.7	119.88	117.62	0.66

Table 10: GPT-4o-mini judge. Per-question-type Recall@10 / answer accuracy (%); *Abst.* is accuracy only (abstention questions have no gold evidence). Among real retrievers, the best and second-best values in each column are highlighted, and the shaded row marks the LightMem default, all-MiniLM-L6-v2. Superscripts indicate a significant difference from the default (a) or from *Oracle* (b), using a paired t -test for Recall@10 and McNemar’s exact test for accuracy. Tests are computed over the questions in each column, with Bonferroni correction applied within each column and anchor family at $p<0.05$. Recall@10 is not tested against *Oracle*, where it is 1.0 by construction. Recall values and their significance are identical to the main table (Table 4); only answer accuracy is re-graded by GPT-4o-mini.

	Retriever	Know-upd.	Multi-sess.	Single-pref.	Single-user	Temporal	Abst.	Overall
<i>Sparse</i>	BM25	0.467 ^a / 77.8	0.300 ^a / 38.0 ^{ab}	0.257 ^a / 33.3 ^{ab}	0.546 ^a / 82.8	0.382 ^a / 55.1	<u>63.3</u>	0.390 ^a / 57.2 ^{ab}
	SPLADE-v3	0.595 / 81.9	0.475 / 66.1	0.641 / 73.3	0.670 / 89.1	0.487 / 62.2	<u>63.3</u>	0.542 / 71.2
	PromptReps-sparse	0.519 / 79.2	0.398 ^a / 47.1 ^b	0.333 ^a / 46.7 ^{ab}	0.603 / 89.1	0.429 / 61.4	<u>63.3</u>	0.456 ^a / 63.5 ^{ab}
	all-MiniLM-L6-v2	0.571 / 79.2	0.464 / 59.5	0.607 / 86.7	0.691 / 89.1	0.474 / 65.4	<u>63.3</u>	0.530 / 70.7
<i>Dense</i>	Qwen3-Emb-0.6B	0.600 / <u>83.3</u>	0.520 ^a / <u>67.8</u>	0.699 / 86.7	<u>0.703</u> / 92.2	0.486 / 61.4	56.7	0.562 ^a / 72.5
	Qwen3-Emb-4B	0.637^a / 84.7	<u>0.523^a</u> / 69.4	0.747 / <u>80.0</u>	<u>0.699</u> / 92.2	0.531^a / 68.5	66.7	0.587^a / 75.5
	Qwen3-Emb-8B	<u>0.618</u> / 81.9	0.532^a / 65.3	0.733 / 76.7	0.712 / <u>90.6</u>	<u>0.510</u> / <u>66.9</u>	60.0	<u>0.583^a</u> / 72.5
	PromptReps-dense	0.541 / 80.6	0.456 / 57.0	<u>0.737</u> / <u>80.0</u>	0.656 / 92.2	0.431 / 60.6	66.7	0.514 / 69.1
<i>Hybrid</i>	PromptReps-hybrid	0.559 / 80.6	0.460 / 57.0	0.490 / 66.7 ^b	0.644 / 89.1	0.470 / 65.4	56.7	0.512 / 68.5
	MiniLM+BM25	0.556 / 80.6	0.434 / 53.7	0.455 ^a / 73.3	0.653 / 89.1	0.476 / 65.4	46.7 ^b	0.502 ^a / 67.3 ^b
	Qwen3-0.6B+BM25	0.566 / 79.2	0.451 / 57.9	0.421 ^a / 63.3 ^b	0.634 / 92.2	0.479 / 63.0	56.7	0.506 / 68.0 ^b
	<i>Oracle</i>	1.000 / 76.4	1.000 / 68.6	1.000 / 96.7	1.000 / 90.6	1.000 / 66.9	83.3	1.000 / 75.5

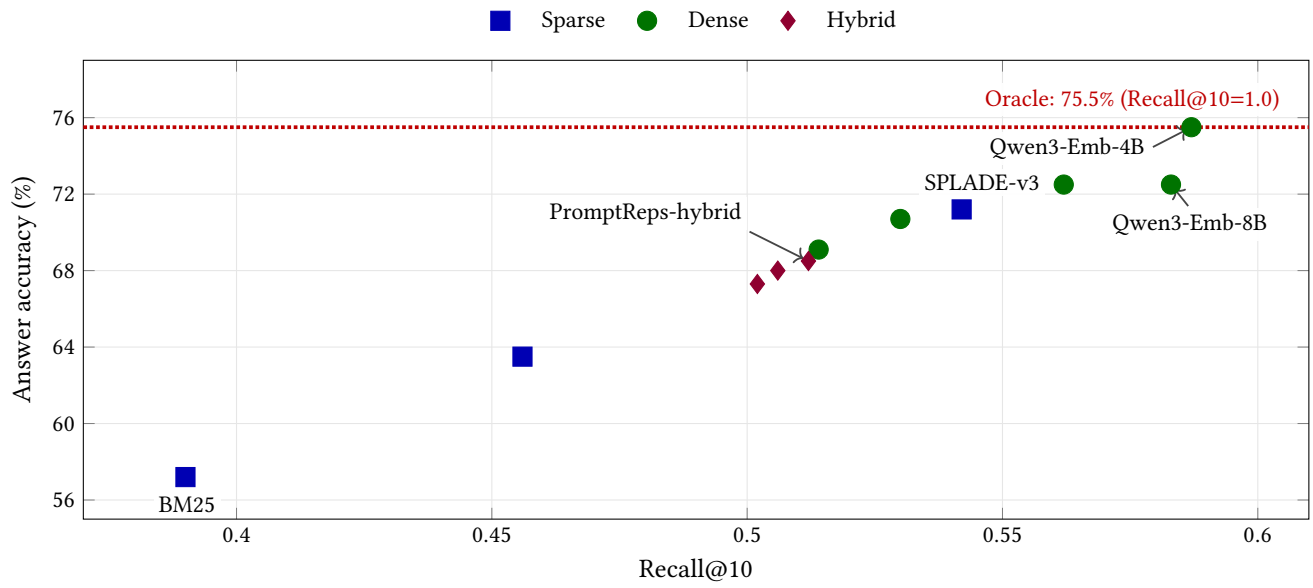


Figure 5: GPT-4o-mini judge. Relationship between Recall@10 and answer accuracy for LightMem at top-10. Each point represents one retriever; colours and markers indicate retriever families, and selected retrievers are labelled. The dotted horizontal line shows the answer accuracy obtained when all memory entries linked to has_answer turns are provided to the generator. Its recall is 1.0 and therefore lies outside the displayed x-axis range. Recall values are identical to the main figure; answer accuracy is re-graded by GPT-4o-mini.

Δ color: ■ LightMem better ■ near parity ■ Naive RAG better darker = larger gap

		(a) Matched retrieval depth			(b) Matched answering-token budgets		
<i>Sparse</i>	Retriever	top-3	top-5	top-10	~330t	~500t	~935t
	BM25	-7.0	-10.6	-10.4	-2.5	-5.4	-2.1
	SPLADE-v3	-9.4	-5.6	-6.5	+1.6	+0.7	-3.2
	PromptReps-sparse	-8.3	-8.6	-6.1	-1.3	-0.5	-1.4
<i>Dense</i>	all-MiniLM-L6-v2	+9.2	+4.9	+3.8	+18.0	+11.0	+6.7
	Qwen3-Emb-0.6B	-4.7	-6.5	-6.1	+2.5	-0.9	-2.7
	Qwen3-Emb-4B	+1.3	-1.1	+0.7	+9.2	+5.7	+2.7
	Qwen3-Emb-8B	-1.5	-2.3	-4.5	+5.2	+1.3	-3.6
	PromptReps-dense	+3.4	+0.7	+2.0	+13.1	+12.8	+4.3
<i>Hybrid</i>	PromptReps-hybrid	-2.5	-6.3	-6.7	+5.8	+0.0	-4.5
	MiniLM+BM25	-2.0	-9.0	-5.4	+6.1	+0.2	-3.6
	Qwen3-0.6B+BM25	-7.9	-11.9	-7.9	-3.9	-2.9	-7.2

Figure 6: GPT-4o-mini judge. Controlled comparison between LightMem and Naive RAG. Each cell reports the answer-accuracy difference $\Delta = \text{LightMem} - \text{Naive RAG}$ in points. Panel (a) matches retrieval depth; panel (b) matches the answering-token budget. Retrievers share the left column and are grouped into sparse, dense, and hybrid families. Answer accuracy is re-graded by GPT-4o-mini.